CSS5280 Design and Analysis of Complex Systems

Lecture 10: Agent-based Models:

- Lab 4 Implementations
- Opinion Dynamics and Platform Policy Modeling

Instructor: Zhanzhan Zhao

School of Humanities and Social Science
School of Artificial Intelligence
School of Data Science (by courtesy)
CUHK, Shenzhen

Lab 4:
Simulate a basic city model

Overview:

- Understanding how individual vs. collective decision rules shape social patterns (e.g., segregation vs. mixing).
- Focus a Python implementation of the Schelling-type model, step by step
- Finally, put all modules together in the notebook to reproduce the key results from the PNAS article (segregation vs. mixing transition).

Learning Objectives:

By the end of this session, you should be able to:

- Explain the role of each Python modules in the simulation pipeline.
- Relate the code functions (Δu, ΔU, α, T) to theoretical concepts from the PNAS paper.
- Run and interpret basic simulations of residential moves.Visualize outcomes (heatmaps, 3D surfaces) and connect them to social science concepts like segregation, cooperation, and externalities.
- Critically discuss: Why self-interested behavior may lead to inefficient collective outcomes; How cooperation (α) and randomness (T) influence system dynamics.

Before starting

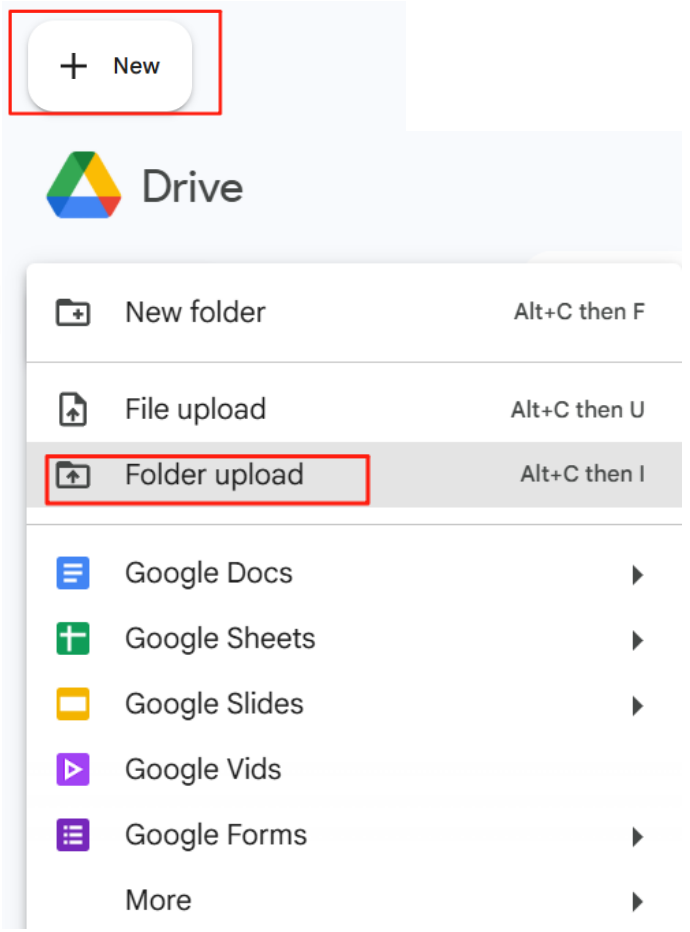
- Please connect to the CUHKSZ Wi-Fi while on campus,
- Please use the school VPN when off campus
- For more details: https://itso.cuhk.edu.cn/node/169

Step 1: **Get lab files**

- In your browser, go to Blackboard.
- Locate **Lab 4** and download the **Lab 4.zip**
- **Find City-simulation in your PC (keep the original filenames and file structure).**

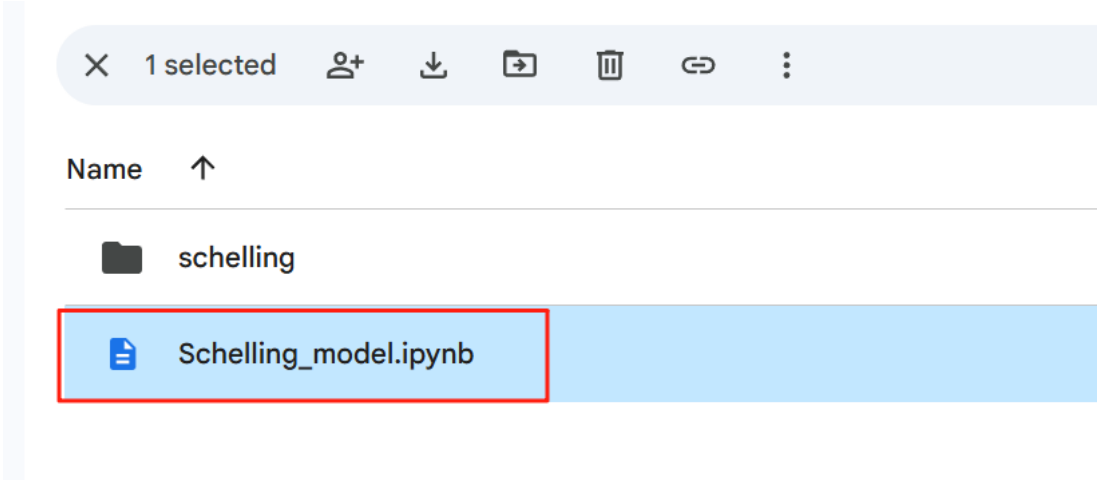
Step 2: Upload Lab files to Google Drive

- Unzip the City-simulation folder you just downloaded
- In your browser, go to: https://drive.google.com/
- Click '+ New' in the upper left corner.
- Click 'Folder upload'
- Upload the City-simulation folder you just unzipped.



Step 3: Open the *ipynb* in Google Colab.

- Double-click the ipynb file.
- It will automatically open in Google Colab.

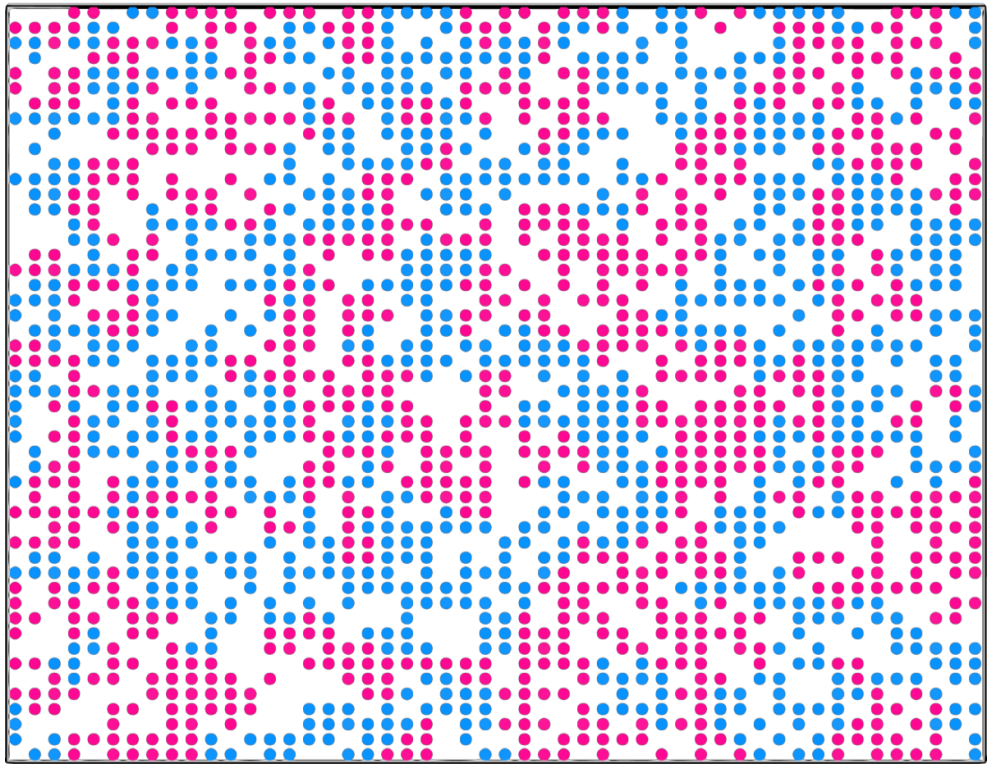


💡 Tip: Using Google Chrome may provide the smoothest experience when accessing Google services.

The Schelling Model of Segregation

- Introduced by **Thomas Schelling (1970s)**
- A **foundational agent-based model** for studying spatial segregation
- Shows how **mild local preferences** can lead to **large-scale segregation**
- Agents of different types (e.g., colors/groups) occupy a grid
- Move if their neighborhood satisfaction threshold is not met
- Despite its simplicity, reveals **emergent collective behavior**
- Widely applied in **economics, sociology, and urban studies**

Schelling Model with two colours: Final State with Happiness Threshold 30%



Segregated neighbourhoods

💡**In this lab:** We will implement and explore key variants of the Schelling model

Parameters Recap

Parameter	Explanation / Analogy
Q	Number of neighborhoods (blocks) → city cut into Q small grids
H	Capacity per neighborhood (max residents) → like H apartments per block
ρ (rho)	Density of a block = residents ÷ H → 0 = empty, 1 = full
ρ_0 (rho0)	Average citywide density → e.g., 0.5 = "half full on average"
u(ρ)	Individual utility at density ρ → symmetric: prefer half full; asymmetric: tilted preference
U	Collective utility = sum of all individuals' utility
α (alpha)	Cooperation / altruism → 0 = selfish, 1 = collective, in-between = mixed
T (temperature)	Randomness / noise → higher T = more chance-based, suboptimal moves possible
m	Asymmetry of utility → larger m = more tolerant of high density, needs higher α to mix

Key Components:

- **Initialization strategies**: how the city and agents are initially distributed
  – *e.g.*, random placement, balanced layout, half-full blocks
- **Agent selection**: which agent moves next
  – *e.g.*, random agent, agent with lowest utility
- **Target selection**: where the agent may want to move
  – *e.g.*, random vacant cell, block with highest utility
- **Move acceptance rule**: when a move is allowed
  – *e.g.*, only if personal utility increases, or social welfare increases, or via Metropolis-Hastings
- **Stopping criteria**: when to halt the simulation
  – *e.g.*, fixed number of steps, or no more accepted moves

Modular Code Organization: schelling/

Our implementation is organized as a **modular package**, where each simulation component is placed in its own file.

This makes the code easier to **understand, maintain, and extend.**

- **initialization.py** – how agents are initially placed in the city
- **agent_selector.py** – strategy for choosing which agent moves
- **target_selector.py** – strategy for choosing where the agent might move
- **move_acceptor.py** – rule for deciding whether a move is accepted
- **stop_criterion.py** – when to stop the simulation
- **utils.py** – shared utility functions and plotting tools

💡 Each module provides a **baseline implementation** that is simple but functional, and can be easily replaced to explore new model variants.

initialization.py — Randomly Fill a City to a Given Average Density

- Given a city divided into Q blocks, each with capacity H, and a target average density ρ_0, the function randomly places people into blocks according to that density.

- Inputs (3 key parameters):
  Number of blocks Q
  Capacity per block H
  Target average density $\rho_0 \in (0,1]$

e.g. Q = 36, H = 100, ρ_0 = 0.5 → Place 36 × 100 × 0.5 = 1800 people

- **Output:** An array occupied of length Q, where each entry records how many people live in that block.

💡**Why random?** This gives us an unbiased starting point. In later models, people will "move." Starting from randomness allows us to compare how different movement rules lead cities to spontaneously form mixed vs. segregated patterns.

```python
import numpy as np


def init_random(Q, H, rho0):
    """
    Randomly initialize a city by placing agents according to a given global density.

    Parameters:
    - Q (int): Number of neighborhoods (blocks)
    - H (int): Capacity of each neighborhood
    - rho0 (float): Initial average density (0 < rho0 <= 1)

    Returns:
    - occupied (np.ndarray): Array of length Q, indicating the number of agents in each neighborhood
    """
    # Total number of agents to place
    N = int(Q * H * rho0)

    # Initialize all blocks as empty
    occupied = np.zeros(Q, dtype=int)

    # Place each agent one by one into a randomly selected block
    for _ in range(N):
        q = np.random.randint(Q)
        # If the selected block is full, keep trying until a non-full block is found
        while occupied[q] >= H:
            q = np.random.randint(Q)
        occupied[q] += 1

    return occupied.copy()
```

agent_selector.py — Deciding “Who Moves Next”

Problem it solves: In each simulation step, we must first “pick” one resident. Which block does this resident come from?

Core logic:

- Blocks with **more people** → higher chance of being selected
- Blocks with **fewer people** → lower chance
- If the city is empty → return None

Input: occupied, an array of length Q, with the number of residents in each block

Output: An **integer index** indicating which block the selected resident comes from

💡Residents in larger blocks are more likely to be picked to move.

```python
import numpy as np

def select_agent_random(occupied):
    """
    Randomly selects an agent uniformly from the whole population.

    Parameters:
    - occupied: np.array of shape (Q,), number of agents in each block

    Returns:
    - index (int) of selected agent's block
    """
    total_agents = np.sum(occupied)
    if total_agents == 0:
        return None  # no agents to select

    # Normalize to get sampling probabilities
    probs = occupied / total_agents

    # Weighted random choice over block indices
    return np.random.choice(len(occupied), p=probs)
```

target_selector.py — Deciding "Where to Move"

Once a resident is chosen to move, we must decide: **Which block do they go to?**

Two strategies:

1. **random_block (uniform over blocks):**
   - Randomly pick from all blocks that are **not full**
   - Number of empty spots does **not** matter
2. **random_cell (weighted by vacancies):**
   - Every empty house has the same chance of being chosen
   - Blocks with more vacancies are **more likely** to be selected

Input: occupied (residents per block)
H (capacity per block)

Output: An **integer index** for the target block, or None if the city is already full

💡**Choose the destination:** either **pick a block at random**,
or **weight by how many empty houses it has**.

```python
import numpy as np

def select_target_random_block(occupied, H):
    """
    Randomly selects a target block from the set of non-full blocks.

    Parameters:
    - occupied (np.ndarray): Array of shape (Q,), number of agents in each block
    - H (int): Capacity of each block

    Returns:
    - index (int) of selected target block, or None if all blocks are full
    """
    candidates = np.where(occupied < H)[0]
    if len(candidates) == 0:
        return None  # no valid target
    return np.random.choice(candidates)


def select_target_random_cell(occupied, H):
    """
    Randomly selects a target block from the set of non-full blocks.

    Parameters:
    - occupied (np.ndarray): Array of shape (Q,), number of agents in each block
    - H (int): Capacity of each block

    Returns:
    - index (int) of selected target block, or None if all blocks are full
    """
    total_cells = H * occupied.shape[0] - np.sum(occupied)
    if total_cells == 0:
        return None  # no agents to select

    # Normalize to get sampling probabilities
    probs = (H - occupied) / total_cells

    # Weighted random choice over block indices
    return np.random.choice(len(occupied), p=probs)
```

move_acceptor.py — Deciding "Move or Stay"

Three levels of decision rules:

1.Individual version — accept_if_personal_utility_improves

Move only if **personal utility increases**: $u(to) > u(from)$

2. Accelerator — build_marginal_table

Speeds up calculation

3.Comprehensive version — accept_metropolis (with α, T)

Compute personal change Δu and social change ΔU

Composite utility combine into: $G = \Delta u + \alpha(\Delta U - \Delta u)$

Zero temperature (T=0): move if G > 0

Finite temperature (T>0): move with probability depending on G and T

💡**Parameters:**

α ("altruism / cooperation"): how much weight to give others' utility

T ("randomness / noise"): higher T → more chance-based moves

```python
import numpy as np

def accept_if_personal_utility_improves(from_density, to_density, utility_fn):
    """
    Accept the move if the agent's personal utility increases.

    Parameters:
    - from_density (float): Current density of the block the agent is in
    - to_density (float): Density of the block the agent is considering moving to
    - utility_fn (callable): Function u(ρ) that returns utility given density ρ

    Returns:
    - True if move is accepted (Δu > 0), False otherwise
    """
    u_current = utility_fn(from_density)
    u_new = utility_fn(to_density)
    return u_new > u_current


def build_marginal_table(H, utility_fn):
    rhos = np.arange(H + 1) / H
    u = np.vectorize(utility_fn)(rhos)
    g = H * rhos * u
    m = g[1:] - g[:-1]
    return m

def accept_metropolis(from_idx, to_idx, occupied, H, utility_fn, alpha=0.0, T=0.01, m_tab=None):
    delta_u = utility_fn((occupied[to_idx] + 1) / H) - utility_fn(occupied[from_idx] / H)
    G = delta_u

    if alpha:
        n_to   = occupied[to_idx]
        n_from = occupied[from_idx]


        if m_tab is not None:
            delta_U = m_tab[n_to] - m_tab[n_from - 1]
        else:
            def g(n):
                r = n / H
                return H * r * utility_fn(r)
            delta_U = (g(n_to + 1) - g(n_to)) + (g(n_from - 1) - g(n_from))

        G += alpha * (delta_U - delta_u)

    if T == 0:
        return G > 0
```

stop_criterion.py — Deciding "When to Stop"

Problem it solves:
The simulation cannot run forever — we need a **stopping condition**.

Strategy used here:

Fixed number of steps

A maximum step count max_steps is given (e.g., 100,000)

At each step:

- If current_step ≥ max_steps → **Stop**
- Else → **Continue**

Input: current_step: current step number;
max_steps: maximum number of steps

Output: True → Stop;
False → Continue simulation

```python
def stop_after_fixed_steps(current_step, max_steps):
    """
    Stop criterion: stop the simulation after a fixed number of steps.

    Parameters:
    - current_step (int): Current simulation step (starting from 0)
    - max_steps (int): Maximum number of allowed steps

    Returns:
    - True if the simulation should stop, False otherwise
    """
    return current_step >= max_steps
```

utils.py (Part 1)—Utility Functions: Scoring "Preference for Density"

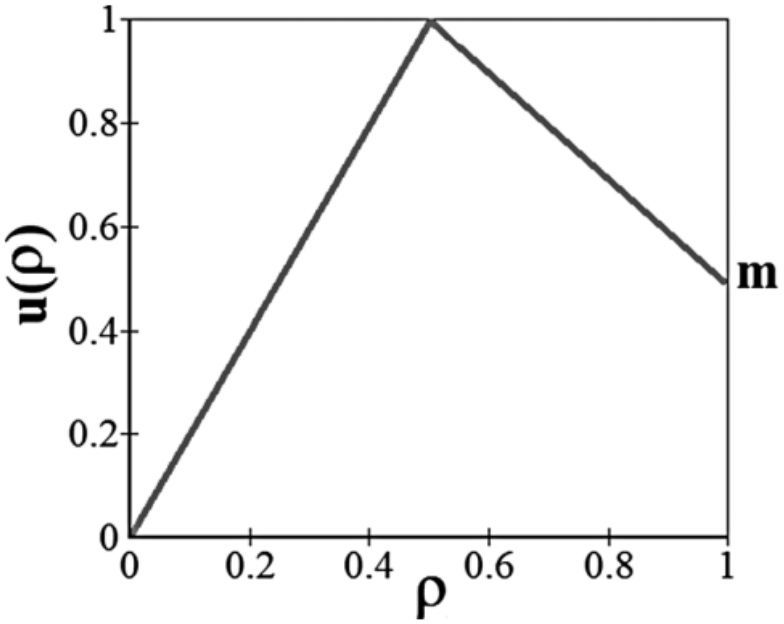
What do we want to express?
For each neighborhood density **ρ**, compute the resident's **satisfaction** *u(ρ)*.

Symmetric triangle — triangle_utility

- Peak at ρ = 0.5
- Closer to 0.5 → happier;
- farther → less happy

Asymmetric triangle — asymmetric_triangle_utility

- Peak position/height adjustable
- End-point values adjustable
- Left slope ≠ Right slope
- Matches **asymmetric preferences** in the paper (e.g., steeper or wider on one side)



💡 **Key idea:** *u(ρ) translates "psychological preferences" into computable numbers.*

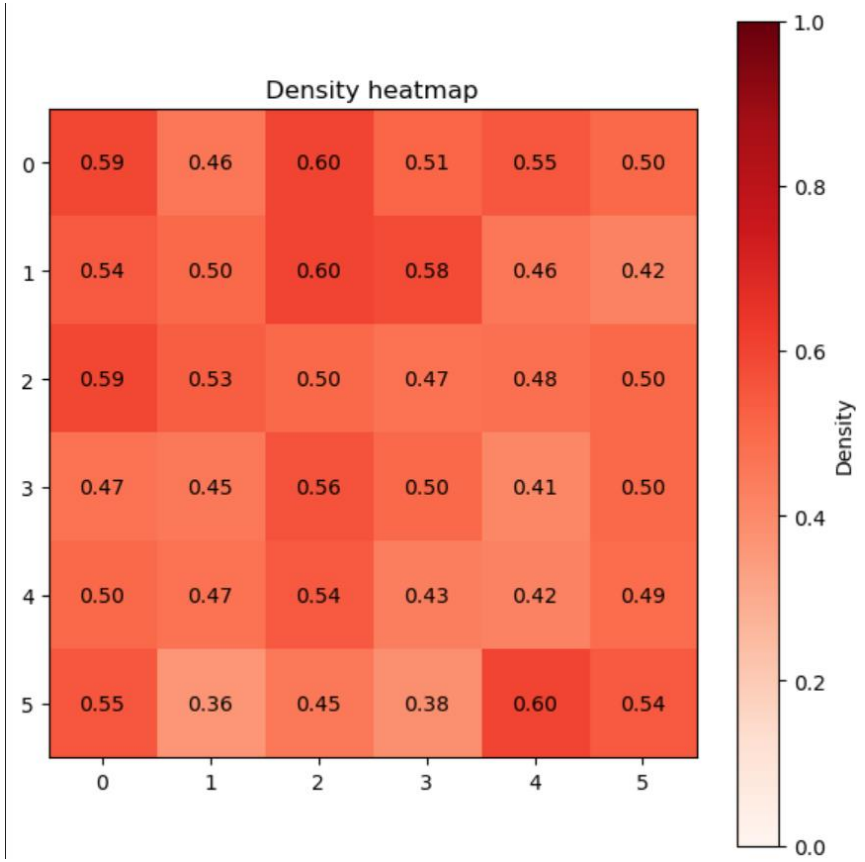
utils.py (Part 2)

—Heatmaps & Line Charts: 2D Views of “Where It’s Crowded, How It Evolves”

1. Heatmap — plot_density_heatmap
Input: grid size (rows × cols) + 1D density array
Output:

- Darker red = more crowded
- Each cell labeled with its value
- Colorbar included

Use case: compare spatial patterns under different **α / T** values



Now, let's turn to the *.ipynb* notebook file.

Notebook Overview (.ipynb)

Purpose:

Combine all the .py modules and run a **complete simulation** end-to-end

Three main parts:

1.Initialization

Set parameters **Q, H, ρ_0**, etc.

Build an "empty city" and randomly place agents

2.Simulation loop

Repeatedly perform: **select source → select target → decide move**

3.Results visualization

Inspect outcomes with **heatmaps, line charts, phase diagrams (3D surfaces)**

💡 **Notebook = our "laboratory bench"**

City Initialization

What it does:

- **Input parameters:** number of neighborhoods Q, capacity H, average density ρ_0
- Randomly assign residents into neighborhoods

Why do this?

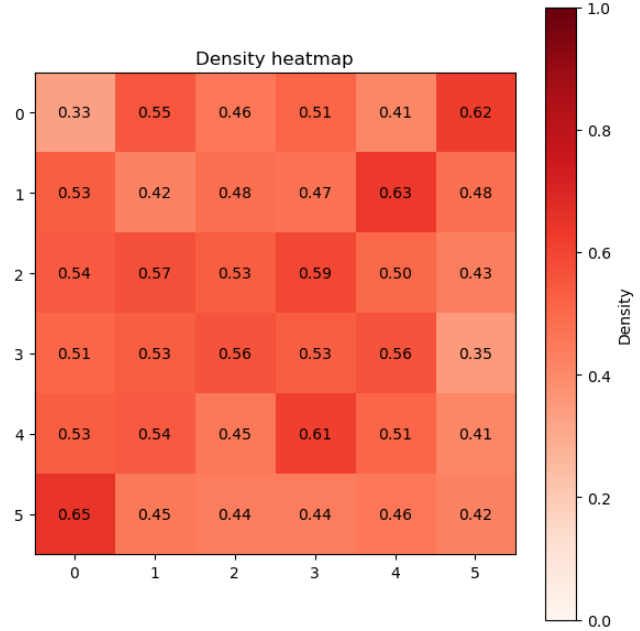
- To give the city an **unbiased starting point**
- Like “scattering beans” randomly into blocks

What can we see?

- Use a **heatmap** to check: which neighborhoods are crowded, which are sparse
- This is the **initial state**, before any dynamic evolution begins

```
# === Parameters ===
Q = 36         # number of neighborhoods
H = 100         # capacity per block
rho0 = 0.5     # initial global density
max_steps = 100000

# === Initialize ===
occupied = initialization.init_random(Q, H, rho0)
utils.plot_density_heatmap(6, 6, occupied / H, title='Density heatmap', xlabel='', yl
```

✓ 0.5s



Baseline: Purely Self-Interested

Purpose:

- A baseline experiment where agents care **only about themselves**
- **Rules:** move **only if personal utility improves** (Δu > 0)

Utility function: symmetric triangle *u(ρ)* (peak at half full)

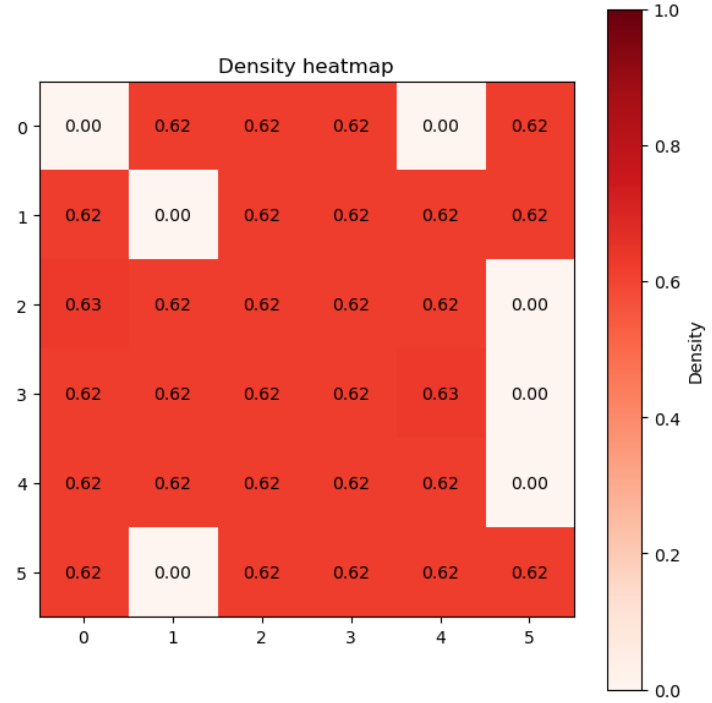
Observed phenomenon:

- City gradually develops **segregated patterns** (some blocks full, others empty) 👉
- Overall collective utility remains **low**

Corresponding to paper:

- **α = 0** (purely selfish)

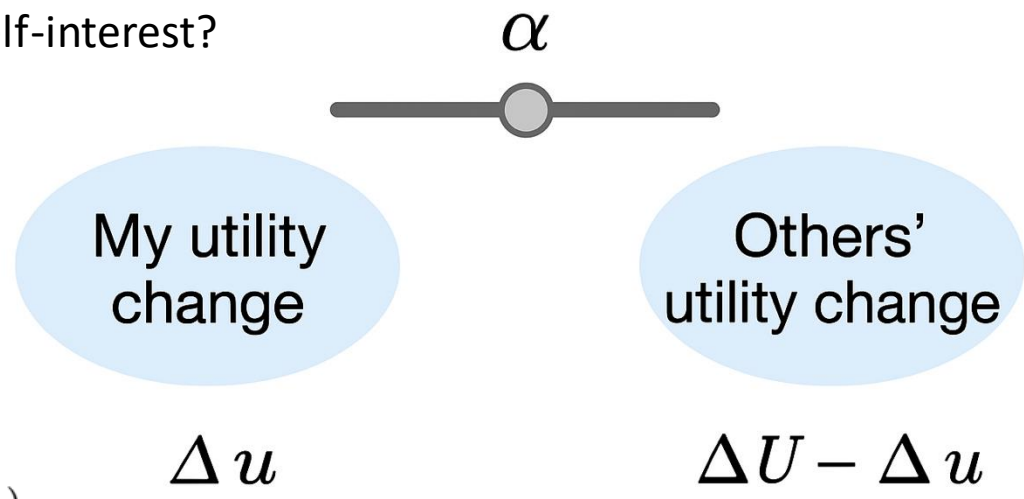
```python
# === Main loop ===
step = 0
while True:
    # Stopping criterion
    if stop_criterion.stop_after_fixed_steps(step, max_steps):
        print(f"Stopped at step {step}")
        break

    # 1. Select an agent block (weighted by number of agents)
    from_block = agent_selector.select_agent_random(occupied)

    # 2. Select a target block (must have space)
    to_block = target_selector.select_target_random_cell(occupied, H)

    # 3. Check if move is accepted (Δu > 0)
    if move_acceptor.accept_if_personal_utility_improves(occupied[from_block] / H, (occupied[to_block]
        occupied[from_block] -= 1
        occupied[to_block] += 1
    step += 1
```



From Purely Selfish to Collective Dynamics

Core question:

•If agents consider **collective utility (ΔU)** in addition to **personal utility (Δu)**,
→ Can we avoid segregation and inefficiency caused by pure self-interest?



Key parameter: α

•**α = 0** → purely selfish
•**α = 1** → fully collective
•**0 < α < 1** → mixed behavior

Connection to the PNAS paper:

•Construct a bridge between Δu and ΔU: $G = \Delta u + \alpha(\Delta U - \Delta u)$

or $G = (1-\alpha)\,\Delta u \; + \; \alpha\,\Delta U$

•Provide a critical threshold formula: $\alpha_c = \frac{1}{3-2m}$

•Explain the **phase transition** from segregation → mixing

What is T?

Temperature (**T**) = randomness / noise in real life, Captures hesitation, mistakes, or “going with the flow”

Case 1: T = 0 (strict rule) 

- If **G > 0** → always move
- If **G ≤ 0** → never move
- Like a perfectly rational, cold “economic agent”

Case 2: T small (close to 0) 

- Most of the time: follow G, occasionally: make “mistakes”:
  - Slightly negative G → might still move (small probability)
  - Slightly positive G → might stay
- Like real people showing hesitation or impulse

Case 3: T large  

- Decisions become highly **randomized**
- Move vs. stay ~ like flipping a coin
- Agents barely care about the size of G, often choose randomly

Try it yourself!

Set different parameters
to observe changes in the plot.

Cheat Sheet: Parameter Effects on Heatmaps

Purpose	Setting	Heatmap outcome	Key point to explain
Baseline segregation	m=0.5, α=0, T=0	Mixture of full blocks and empty blocks	Micro-level rationality → macro-level inefficiency
Below threshold	m=0.5, α=0.1, T=0	Segregation	α not beyond α_c
Cross the threshold	m=0.5, α=0.6, T=0	Becomes uniform (≈0.5 each block)	Phase transition: increasing α leads to mixing
Larger m	m=0.8, α=0.3 vs 0.8, T=0	0.3 → segregation, 0.8 → mixing	Higher asymmetry m raises α_c
Increasing T	α=0, T=0/0.1/0.8	From segregation to more uniform	Noise smooths patterns ≠ higher utility
Different ρ_o	ρ_o=0.3, α=1, T=0	Half-full blocks + empty blocks	With limited population, “best mix” is half-full + empty
System size	Q=16 vs Q=100	Coarse-grained vs fine-grained patterns	System size changes resolution, not mechanism

Extension 1:
Studies on Heterogeneity and City Policy Impacts

Overview:

- Explore two extensions of the Schelling model — one adds altruistic agents, the other introduces a city planner.
- Altruistic model: even a few altruists can catalyze a shift from overcrowded segregation to a balanced, half-filled state.
- Planner model: policy investment can reshape neighborhood attractiveness, sometimes leading to stronger centralization or segregation.

Step 1: Get lab files

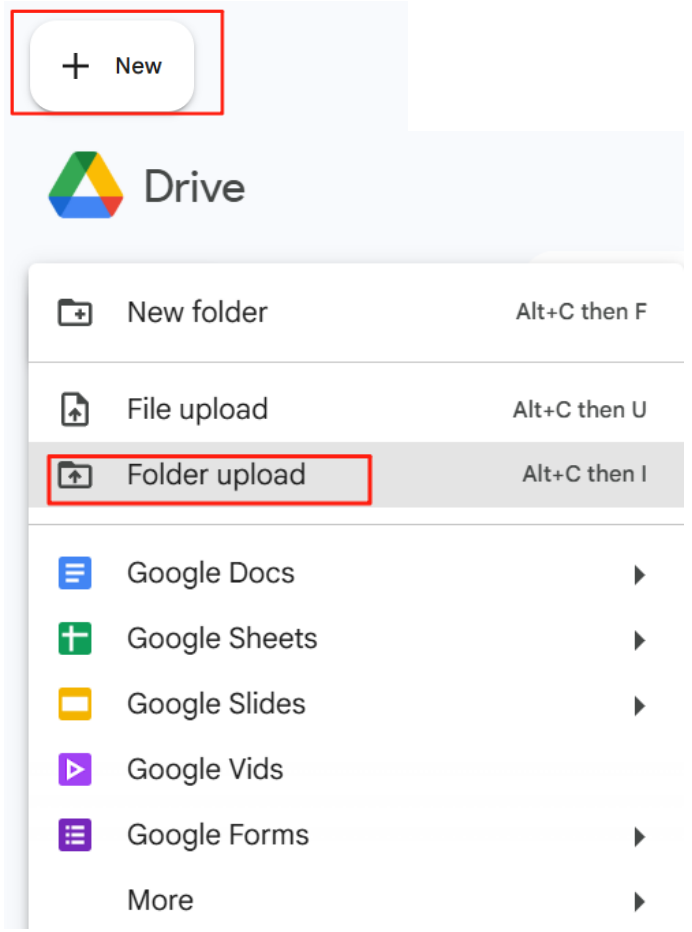
1. In your browser, go to Blackboard.

2. Locate **Lab 4** and download the folder

3. **Unzip City-simulation-Altruists and the City-simulation-Investment these two files in your PC (keep the original filenames and file structure).**

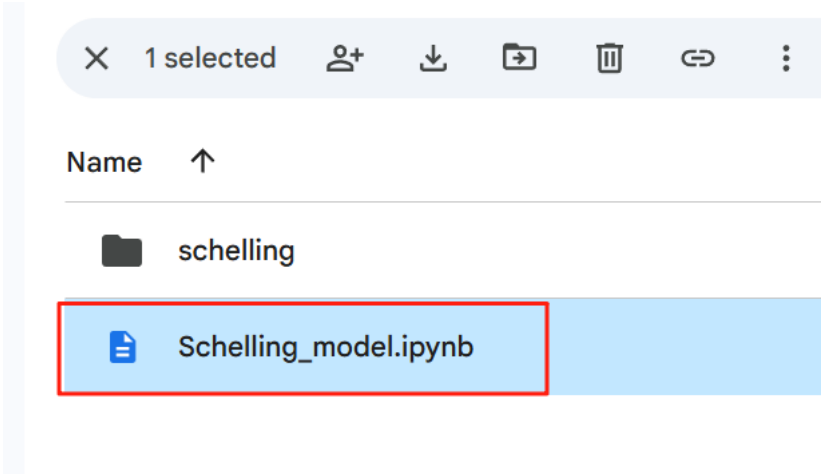
Step 2: Upload Lab files to Google Drive

1. In your browser, go to: https://drive.google.com/
2. Click '+ New' in the upper left corner.
3. Click '**Folder upload**'
4. Upload the two folders **one by one** you just unzipped.



Step 3: Open the *ipynb* in Google Colab.

- Double-click the ipynb files.
- It will automatically open in Google Colab.



💡 Tip: Using Google Chrome may provide the smoothest experience when accessing Google services.

Part 1

Giant Catalytic Effect of Altruists

1. Background and Research Question

Classic Schelling Model

•Agents are **egoists**, moving only to **maximize personal satisfaction**.

•Leads to **excessive segregation**, even when everyone prefers mixed neighborhoods.

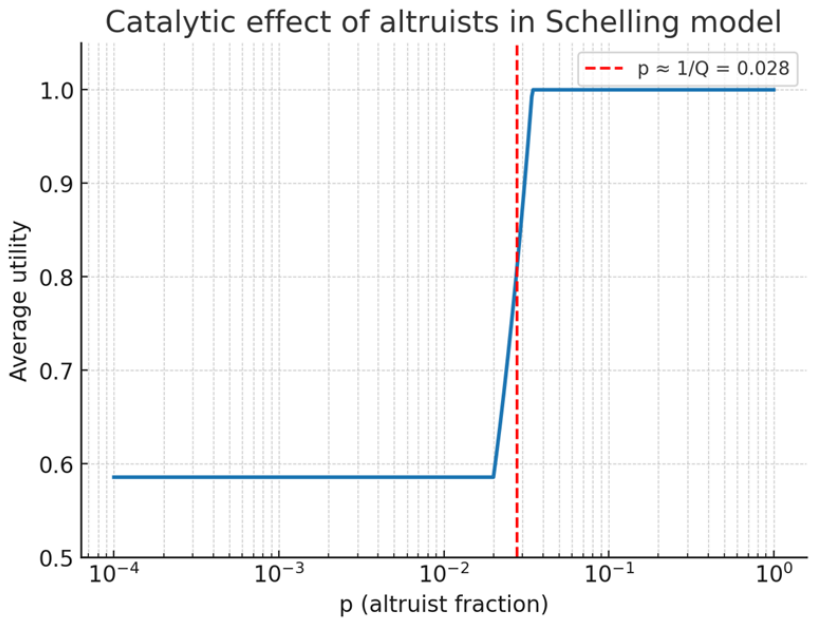
Extension in This Study

•Introduces a **tiny fraction of altruistic agents** who move to **improve collective utility** rather than individual gain.

Core Research Question

•What happens to collective outcomes when a small fraction of altruists is introduced into a population of egoists?

•Can this small group trigger a **"catalytic effect"**—a sudden transition of the entire system toward a socially optimal state?



2. Model Structure and Key Assumptions

1. City and Agents

•The city is divided into **Q blocks (neighborhoods)**, each with a capacity of **H agents**.

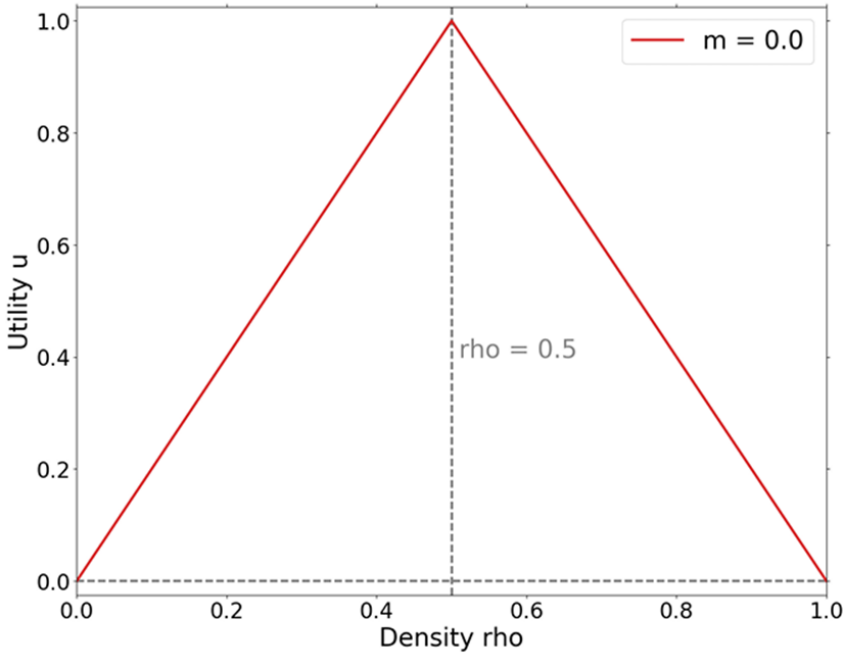
•The **overall density** is ρ_0, representing the initial fraction of occupied sites.

•Each block's density is $\rho_q = N_q / H$

2. Utility Function

$$u(\rho) = \begin{cases} 2\rho, & \rho \leq 0.5 \\ 2(1-\rho), & \rho > 0.5 \end{cases}$$

Agents are happiest when their neighborhood is half full — neither **too empty nor too crowded.**



3. Behavior Rules

•**Egoists:** Move only if their **own utility increases** (Δu > 0).

•**Altruists:** Move only if the **total system utility increases** (ΔU > 0),

where $U = \sum \rho_q u(\rho_q)$

3. Code Structure Overview

1. Initialization

•Randomly generates **two types of agents** — egoists and altruists.

•These two groups will later follow **different moving rules** in the simulation loop.

```python
if (agents is None) and (occupied is None):
    occupied, agents = initialization.init_random_with_types(Q, H, rho0, p)
```

2. Main Loop Logic

•Randomly selects **an agent and a target block** to move to.

```python
idx, from_block, agent_type = agent_selector.select_agent_random_with_types(agents)
while True:
    to_block = target_selector.select_target_random_cell(occupied, H)
    if to_block != from_block:
        break  # avoid self-move
```

3. Movement Rules

①**Egoist:**

•Moves **only if personal utility increases** (Δu > 0).

②**Altruist:**

•Calculates the change in **global utility,** Moves **only if ΔU > 0**.

$$U = \sum \rho_q \, u(\rho_q).$$

```python
if move_acceptor.accept_if_personal_utility_improves(
    occupied[from_block] / H,
    (occupied[to_block] + 1) / H,
    utils.triangle_utility
):
```

```python
delta_U = (U_from_after + U_to_after) - (U_from_before + U_to_before)

if delta_U > 0:  # move only if global welfare increases
    occupied[from_block] -= 1
    occupied[to_block] += 1
    agents[idx][0] = to_block
```

Updates both the occupancy and agent position after each move. The simulation stops after a fixed number of steps.

4. Simulation and Results Analysis

1. Parameter Settings

- The city has 100 neighborhoods, each holding up to 100 agents.
- The initial density is 40%, and 10% of agents are altruists.

```
Q = 100        # number of neighborhoods
H = 100         # capacity per block
rho0 = 0.4     # initial global density
max_steps = 10000
p = 0.1 # fraction of altruistic agents in the population
```

2. Key Metric: Social Welfare

$$U = \sum \rho_q \; u(\rho_q)$$

- Measures the **overall happiness** of the system.
- The closer each block's density is to **half full**, the higher the total welfare.

3. Experiment Loop and Visualization

```
    social_welfares.append(np.sum(occupied * utils.triangle_utility(occupied / H)))
    steps.append((i + 1) * 1000)
utils.plot_line_chart(steps, social_welfares, title="Social welfare over time", xlabel="Steps", ylabel="Social welfare")
```

- Every few steps, compute and record social welfare.
- Plot "Social Welfare over Time" to show how the system evolves and stabilizes.

5. Experimental Findings

Main Results

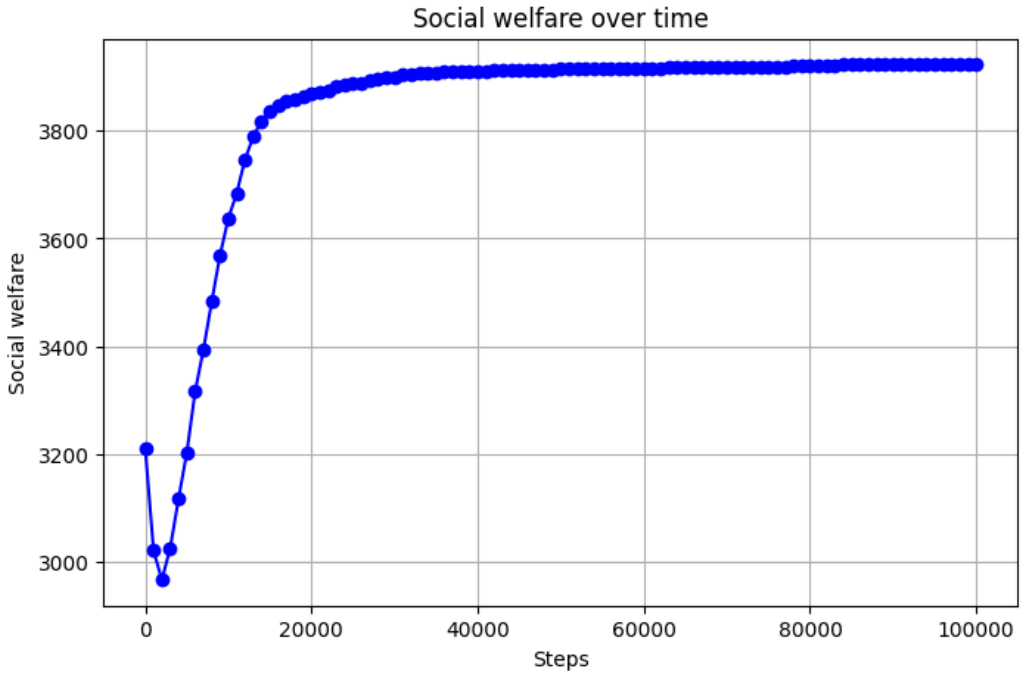
Even with a **small fraction of altruists (low p)**, the system shifts from the **overcrowded state** created by egoists to the **optimal configuration**.

The response is nonlinear:

the improvement in utility is **nonlinear**, but happens through a **phase transition** — a sudden jump in social welfare.

Mechanism:

Altruists act as **catalysts** — their movements open new migration paths, guiding egoists to redistribute themselves and reach a better collective outcome.



Part 2

What if city planners could invest in certain blocks?

Introducing a City Planner (Policy Intervention)

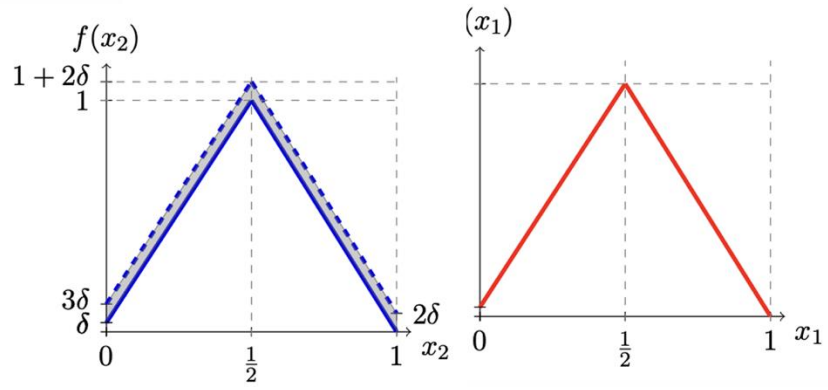
1. Model Extension Idea

•A new **macro-level agent** is added — the **City Planner**.

•The planner can **"invest"** in specific neighborhoods to modify their **utility functions**, making some areas **more attractive** to residents.

2. Mathematical Expression

$$g_q(\rho_q) \geq f_q(\rho_q)$$



•Here, $f_q(\rho_q)$ is the original utility of block q,
and $g_q(\rho_q)$ is the **adjusted utility** after policy intervention.

•The planner's goal is to guide the system toward a **more balanced configuration**.

Code Structure for the Planner Version

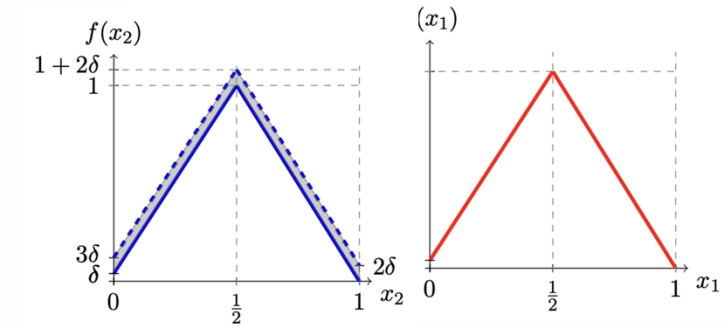
1. Simplified Parameters

•The model is simplified to **2 Blocks** to clearly show the planner's effect.

2. Independent Utility Functions

•Each block has its own **utility function**:

- **Block 1:** slightly improved (e.g., +0.1 reward)
- **Block 2:** remains unchanged




```
utility_funcs = []
utility_funcs.append(lambda x: utils.asymmetric_triangle_utility(x, right_y=0) + 0.1)
utility_funcs.append(lambda x: utils.asymmetric_triangle_utility(x, right_y=0))
utils.plot_density_heatmap(1, 2, version_3(Q, H, rho0, 500, utility_funcs) / H)
```

investment

3. Movement Rule

$$u_{to} > u_{from}$$

•An agent moves **only if the target block's utility is higher** than the current one.

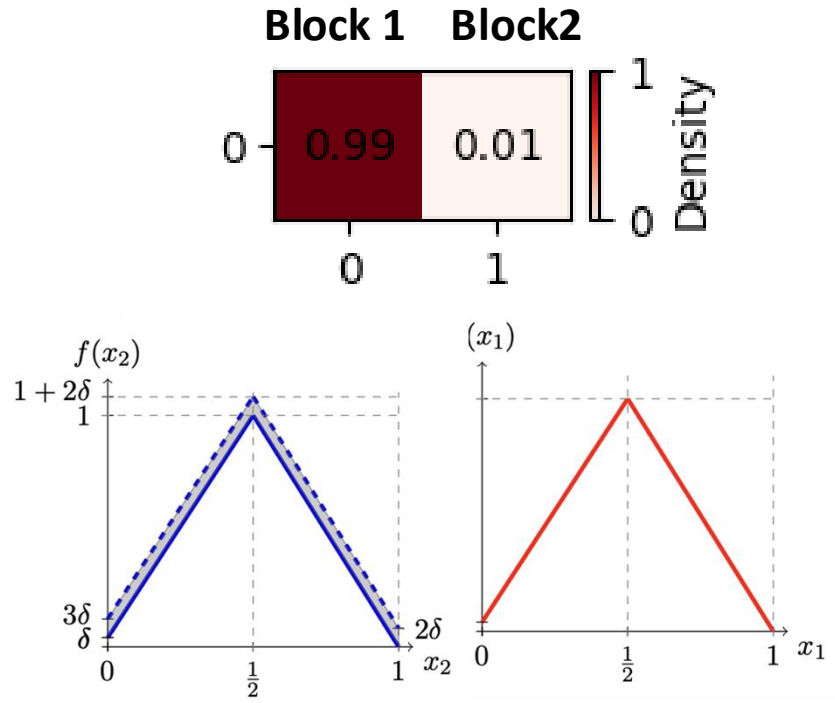
Simulation Result

Observation

•After running the simulation, the **planner's investment** makes **Block 1** much more attractive.

•As a result, most agents move toward Block 1 — forming a **single dominant center**.

•Instead of reducing segregation, the policy actually **intensifies it** — the city becomes **even more unevenly segregated**.



Try it yourself!

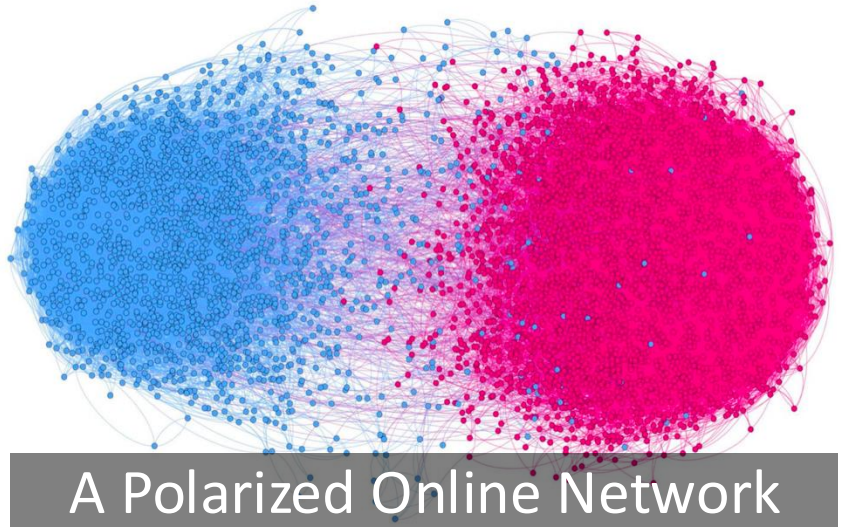
Set different parameters
to observe changes in the plot.

Lecture 9-2:
Opinion Dynamics and Platform Policy Modeling

Ideological Polarization and Recommendations

Recommendation Designs

Influence?



A Polarized Online Network

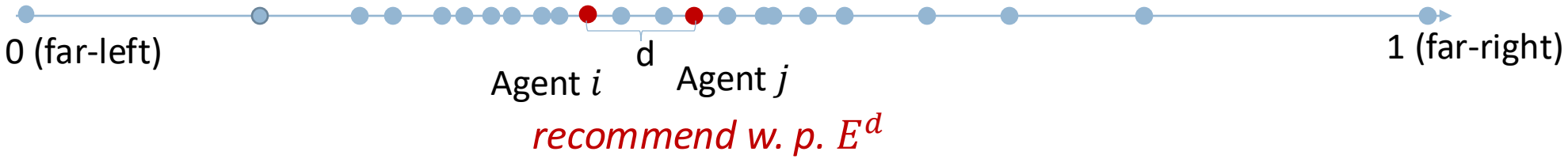
Careful treatment of diversity

bridge

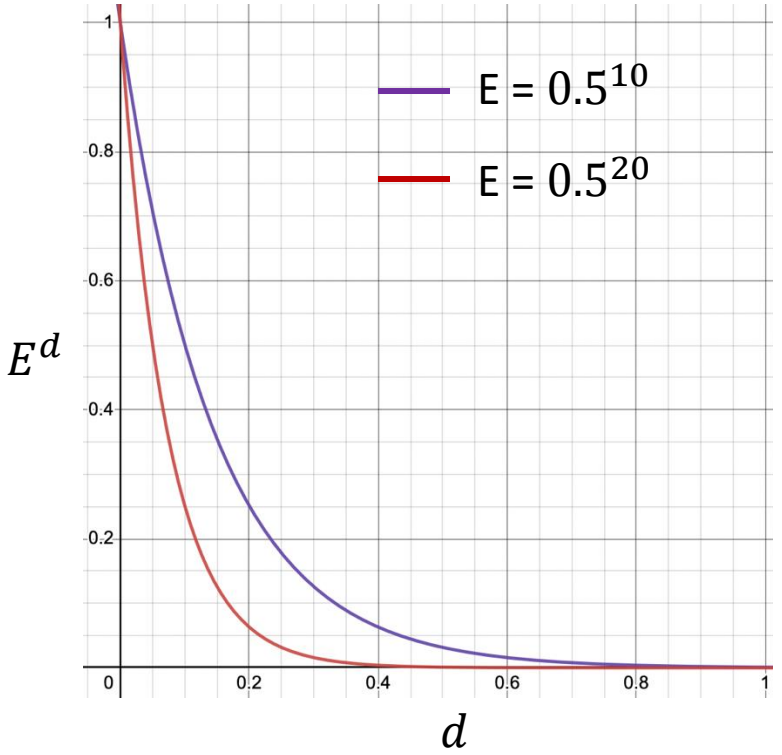
Ideological divides

Modeling Personalized Recommendations *[Axelrod et al., PNAS, 2021]*

- more likely to be recommended with similar-minded others

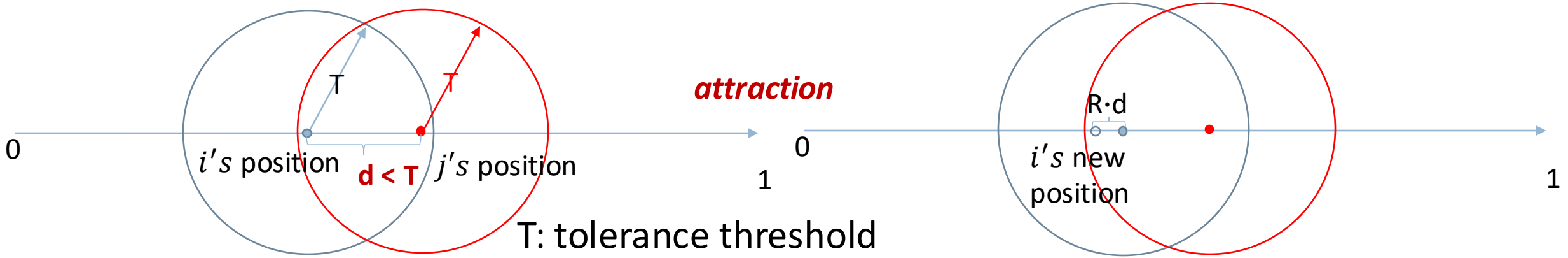


- Personalization parameter E ($E \leq 1$)
  - Smaller E: more personalized
  - Bigger E: more diverse
  - E $\sim 0.5^{20}$: ultra-personalized



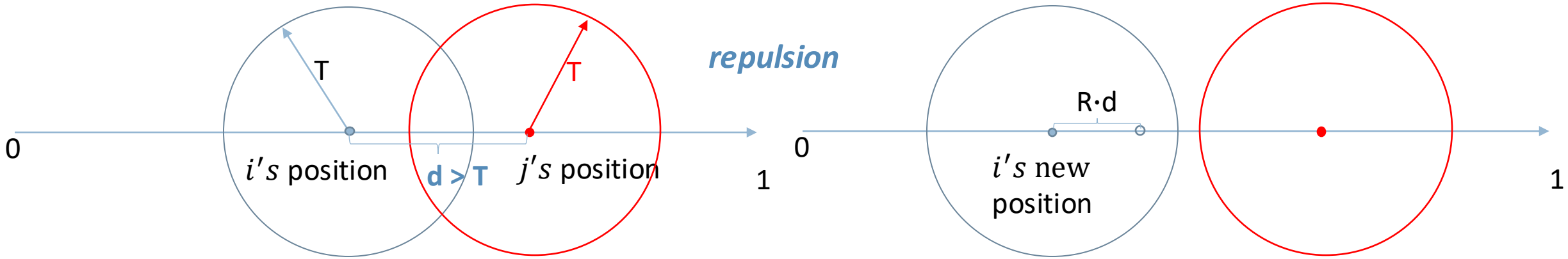
Attraction-repulsion Model *[Axelrod et al., PNAS, 2021]*

- Attraction: Local diversity can lead to comprise



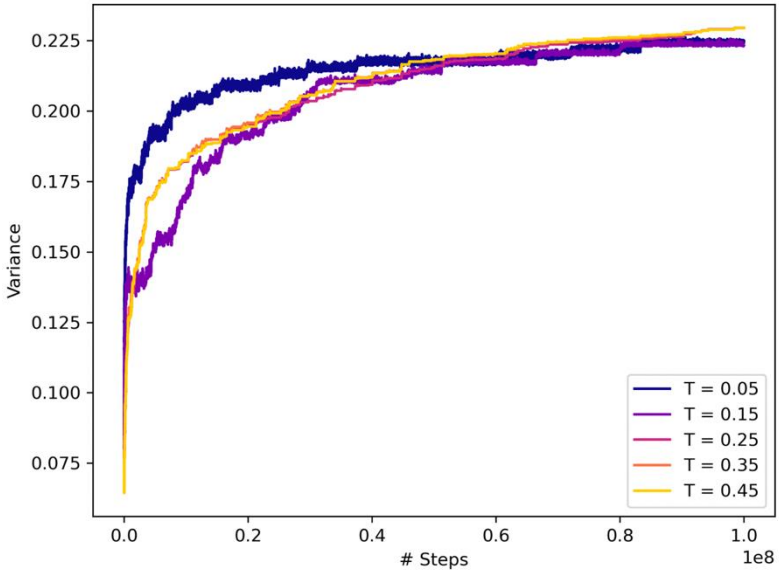
T: tolerance threshold

- Repulsion: Local diversity can lead to conflicts



Ultra-Personalized Recommendations Not Enough

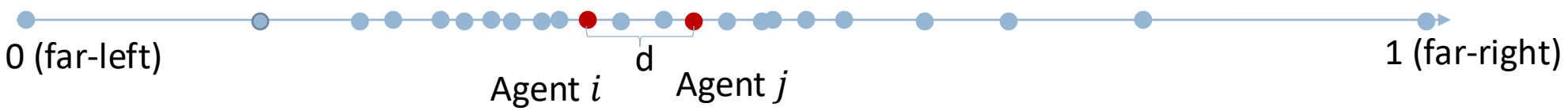
- Initial polarized distribution $G(0.3, 0.1)\ and\ G(0.8, 0.1)$ (our work)



Even ultra-personalized recommendations E $\sim 0.5^{20}$ would increase polarization over time

Tolerance-aware Recommendations' Better Deceleration Effect

Dampen the recommendation probability when beyond agent's tolerance



recommend w. p. E^d if $d \leq T$

recommend w. p. $\frac{E^d}{E'}$ if $d > T$

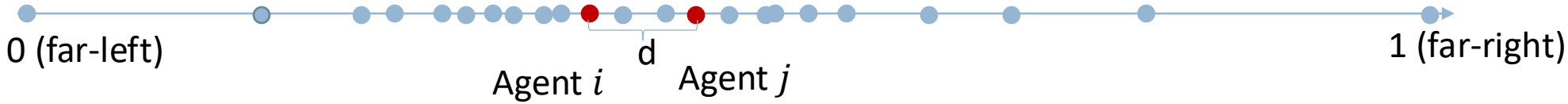
The Better **Deceleration** Effect Under Initial Polarization Condition

[Zhao., Thesis, "Understanding and Mitigating Bias in Algorithms, Data, and Society." 2023]

Tolerance-aware + Adaptively Diversifying's Reversion Effect

Tend to personalize recommendations for agents near middle-ground

Tend to diversify recommendations for more extreme agents



recommend w. p. $(D^{|x_i - 0.5|})^d \; if \; d \leq T$

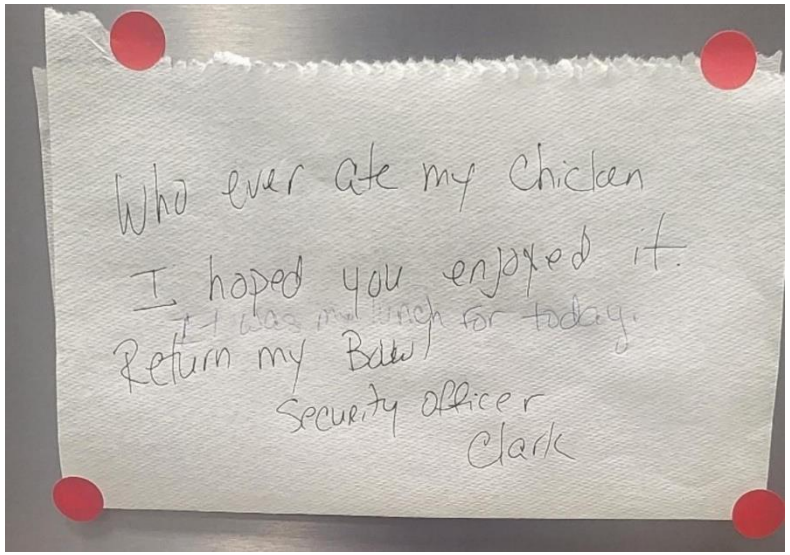
recommend w. p. $\frac{(D^{|x_i - 0.5|})^d}{E'} \; if \; d > T$

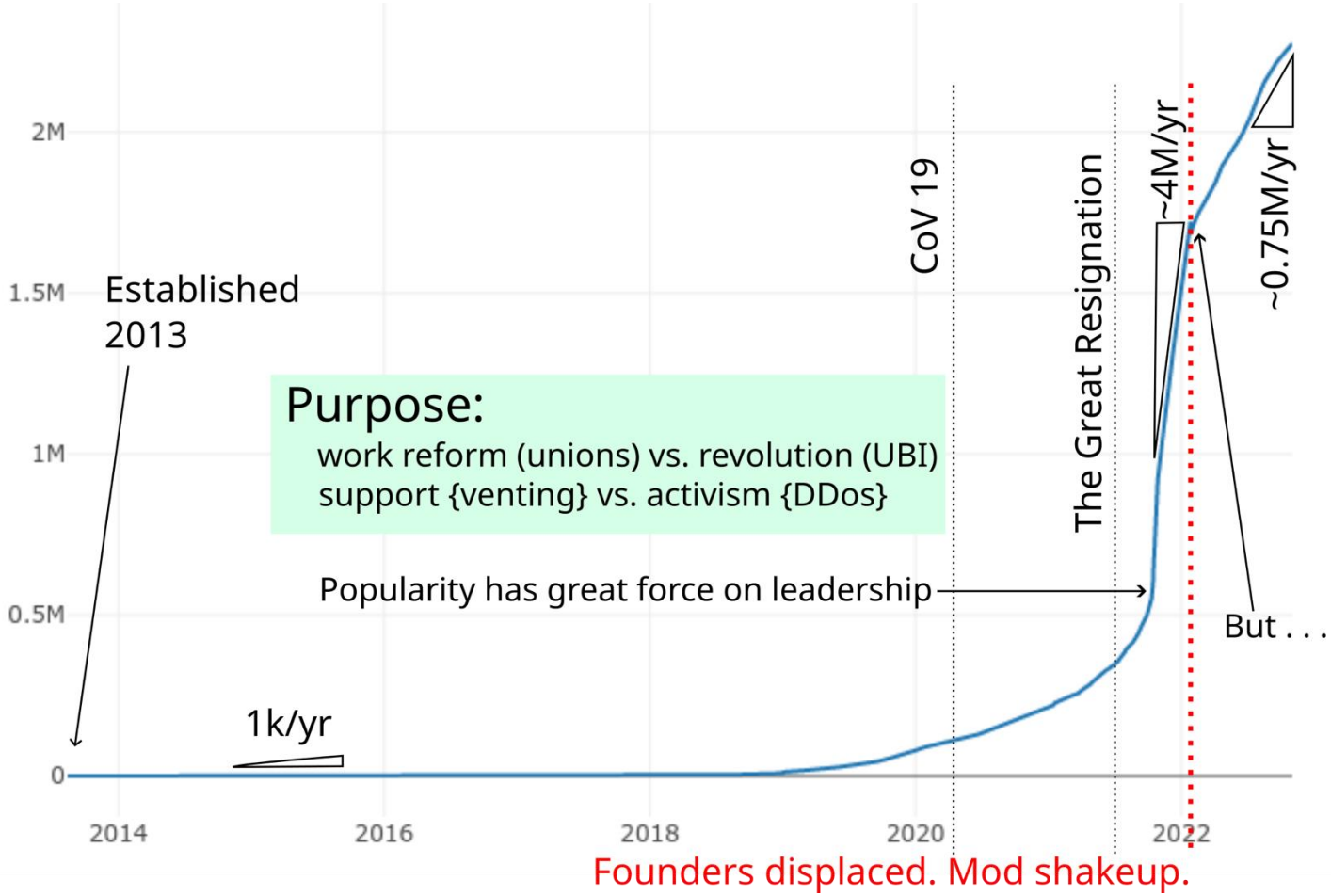
The **Reversion** of Polarization and **Enhancement of Local Diversity**

[Zhao., Thesis, "Understanding and Mitigating Bias in Algorithms, Data, and Society." 2023]

Field Studies

r/antiwork

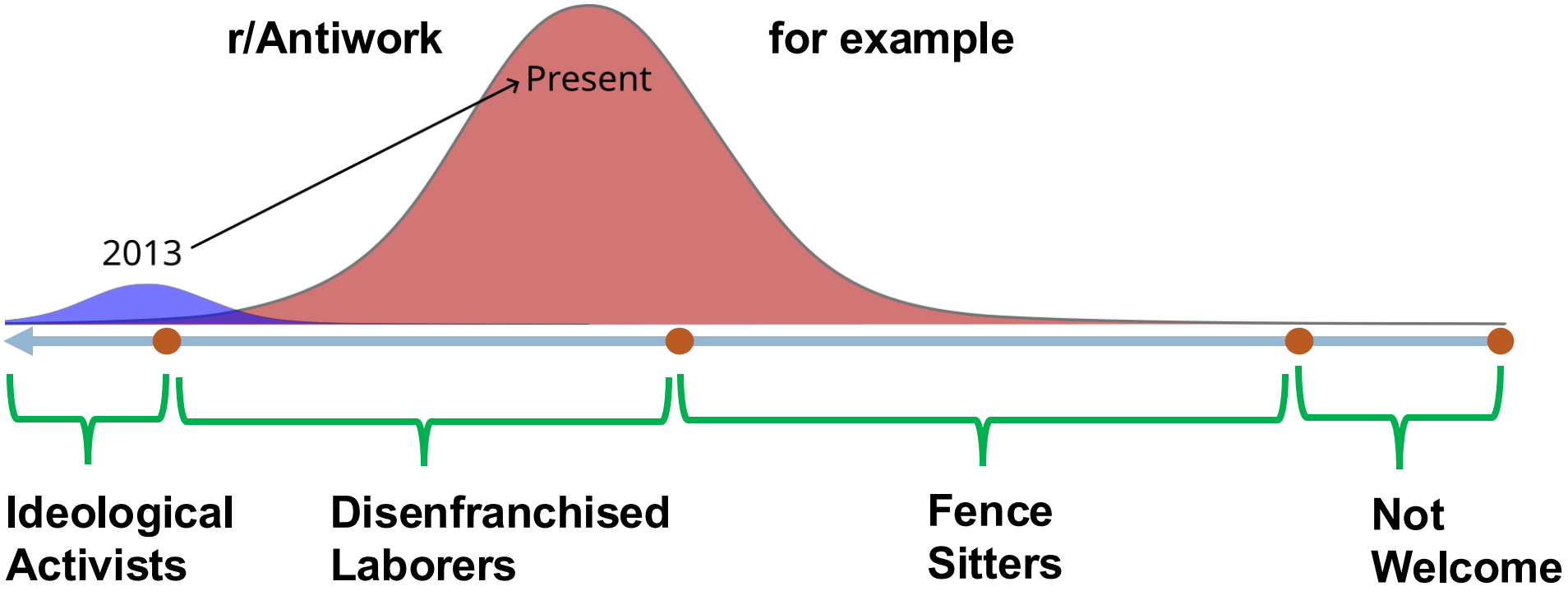




- r/antiwork: over 2 million members, top 10 most popular subreddits
- Ideology dilution: far left -> more moderate, as it gains more popularity recently
- 40-hr participation observations + 13 member interviews (1-2hr each)

Work by Zhanzhan Zhao, Jason Wu, Tanya Murali and Nikith Hosangadi.

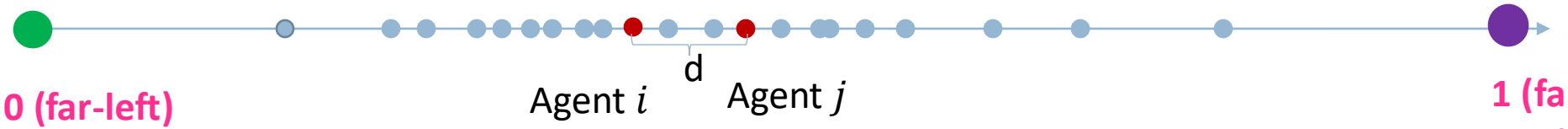
Internal Heterogeneity and The Spiral of Silence



The Spiral of Silence *[E. Noelle-Neumann]*

people are less likely to express their opinions if they believe that their views are not in line with the majority.

The Spiral of Silence and Robustness of the Proposed Design



recommend w. p. E^d

interact w.p. $\boldsymbol{min\left(E^d,\ S^{|x_j - x_{(*,j)}|}\right)}$

$x_{(*,j)}$***: agent j's perceived mainstream position***

Careful treatment of diversity —bridge→ Ideological divides

[Zhao., Thesis, "Understanding and Mitigating Bias in Algorithms, Data, and Society." 2023]

Possible Work: Linking Public Opinions with Digital Policies

Possible Objective: Better model public opinion patterns in China; assess social platforms' effects on public opinions; propose algorithm enhancements.

Possible Questions to study:

How can experiments be designed to validate the effectiveness of our recommendations?

In what ways does the architecture of social platforms shape public opinions?

How can we enhance recommender algorithms to prioritize social welfare and be more cognizant of social dynamics?